

Kubernetes

Configuration basics

ALLIE Arthur, Data Engineer
09 01 2026

Table of content

- 1. Kubernetes and Containers
- 2. Pods
- 3. Pod Networking
- 4. Pod Lifecycle
- 5. Services
- 6. Internal vs External Services
- 7. Ingress
- 8. ConfigMaps and Secrets
- 9. Volumes and Persistence
- 10. Replication and Deployments
- 11. StatefulSets
- 12. Master Node and API Server
- 13. Configuration Files
- 14. etcd
- 15. Infrastructure as Code
- 16. Minikube and kubectl
- 17. Labels and Deployments
- 18. Deployment Selectors and Pod Matching
- 19. Services, Ports, and Target Ports
- 20. Environment Variables and Secrets
- 21. Application-to-Database Configuration
- 22. Clean and Secure Configuration
- 23. External Access with NodePort
- 24. Deployment Order and Dependencies
- 25. Focus on a typical Kubernetes architecture

1. Kubernetes and Containers

1. Kubernetes Pod creates a running environment or a **layer on top** of the **container**.
 - Because Kubernetes: abstracts container technologies (Docker, Podman, etc.).
 - Single entripoint**: Kubernetes (not Docker, Podman, etc.)
 - Don't talk with anything else, **only interact with K8s layer**, on top of everything.
2. Can use below : Docker, Podman, Containerd or any other technology.

Kubernetes, also known as K8s, is an open source system for automating deployment, scaling, and management of containerized applications. → Container orchestration tool.

2. Pods

1. A **Pod** = the **smallest deployable unit** in Kubernetes.
(Essentially a wrapper around one or more containers.)
2. Pod usage: (**usually**) **hosts a single container**.
 - Can run multiple containers inside one pod but usually no.
3. Only case using a **pod** for **many containers** → having a **main application container** inside the pod with:
 - its own helper / side container service.
 - needed inside this pod (to run the main application container).
4. Running a K8s Pod is similar to "running a container inside another container".

3. Pod Networking

1. Each pod gets a **unique & internal IP address** used to **communicate with other pods within the cluster**.
2. **All K8s pods communicate with each other in the K8s cluster:**
 - using this **unique IP address** (and a virtual environment).
 - These **IPs** are **internal, not public**.
3. This **internal IP address** = **specific to the pod**, not to the containers inside.

4. Pod Lifecycle

1. **Pods** are **ephemeral** – they can **crash** or terminate very easily:

→when the **node** (where the pod is running on) is **out of resources**.

→when their containers fail.

→when the app inside the container just crashes.

→*Consequence: their **IP addresses** are **ephemeral too** (change each time we restart a dying pod).*

2. When a **pod restarts**:

→a new one is created at its former place.

→it gets a **new (unique & internal) IP address**.

3. Therefore, pod IPs are temporary and cannot be hardcoded.

5. Services

1. To handle pod volatility, Kubernetes uses **Services**.

→To avoid fixing the ephemeral pod IP address in the configuration file (pod can die at any moment).

2. → Service is like a **stable / permanent IP**→can be **attached to each pod**.

→**Remains constant** even when the **pod dies** & is replaced.

→*Consequence : The lifecycle of services and pods are not connected.*

3. Each **application** or **database** typically has its **own service**.

→Each pod / container / app will have its **own service & static - unique** IP address.

4. Services act as **stable endpoints** that **route traffic to pods without** requiring **IP updates**.

→*Consequence: even if a pod dies, the **service** or its **own ip address** will **stay**.*

6. Internal vs External Services

1. You want your application / service to be **callable externally** (through a browser).

→ You need to have an **external IP address**.

→ Consequence: for that purpose → need to create an **External Service**.

2. **External Service** = service which open the communication **from external sources**.

→ Open communication **from external IPs**, while **internal** ones **restrict access within the cluster**.

→ e.g. *NodePort*.

3. But you don't want your db to be exposed to the public requests (of external users):

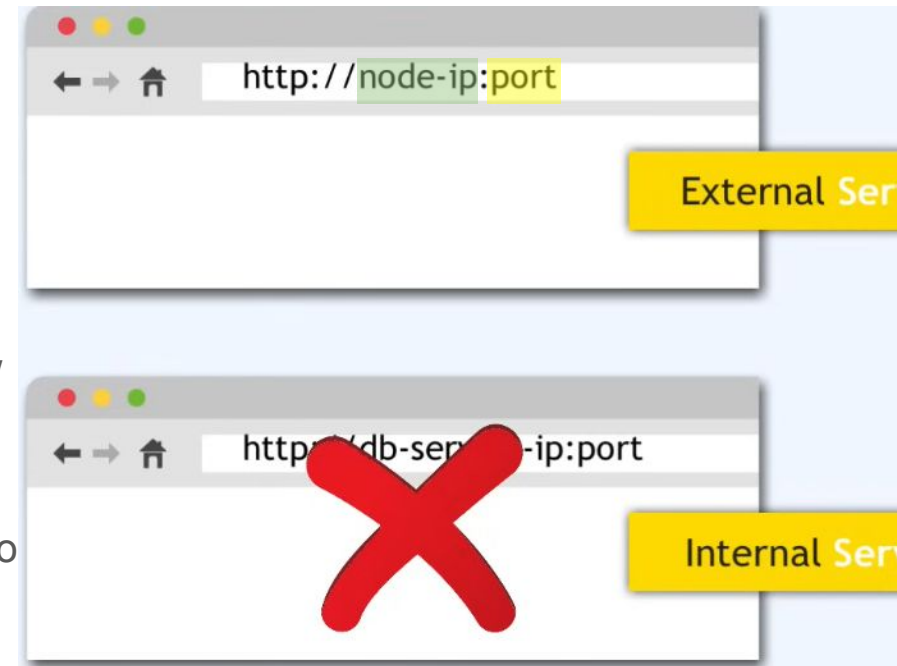
→ Have to create an **Internal Service** (to specify when creating one).

→ This is the **default type** (*ClusterIP*) of a service.

4. **Internal Services** used for components like **db** that **shouldn't be exposed publicly** / **external services** allow **access from outside** sources (e.g., **browsers**).

5. → **IP address** = **IP address of the node** (the service & its pod replicas are running on)

→ **port** = the port number of the service.



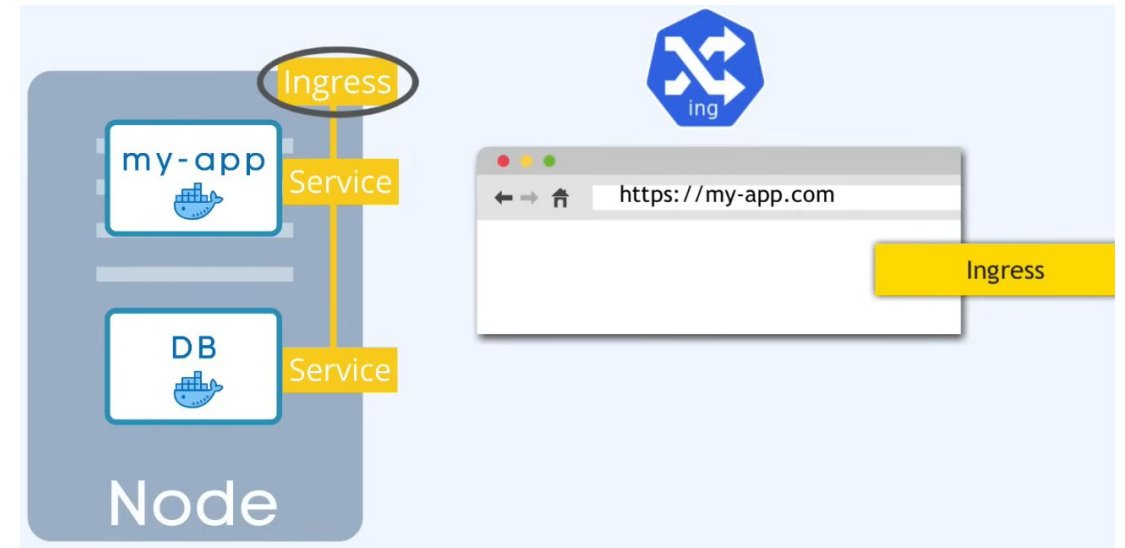
7. Ingress

1. In production: want to have a real (**domain**) name with a secure protocol for your external service (instead of IP address of the node).

- For that, there is another component called **Ingress**.
- Ingress provides a higher-level entry point for external traffic.
- Typically used with **domain names** and HTTPS in production environments.

2. The process – instead of connecting directly to a Service:

- Request **pass first** through **Ingress** (not to the Service).
- Ingress routes / forwards the request to the appropriate Service.



8. ConfigMaps

1. Usually, the Service URL (e.g. the database URL)→configured *inside the built image*.
 - When having to change this URL name: have to *rebuild*→*repush*→*repull* entire new image (with the change inside).
 - to get the new URL name outside of the container.
 - Tedious for a small change*.
2. For that kind of purpose→use a component called **ConfigMaps**.
 - Configmap = **external configuration** for your application.
 - Configmap usually contains some *configuration data information* like an *url*.
3. In Kubernetes: just connect this ConfigMap to the pod.
 - So the pod can access the data inside the ConfigMap.
4. In a nutshell: **ConfigMaps** store **non-sensitive** configuration data externally (like URLs)
 - So don't need to rebuild & redeploy containers for minor config changes.

8'. Secrets

- 1 . Another part of external configuration data = *username & password*.
 - **Can change** during the deployment application process.
 - Putting that data in a plain text format, (even if in an external configuration file) → **unsecure**.
2. For that very purpose → another component called **Secrets**.
3. Difference between Secrets and ConfigMaps:
 - Secrets is dedicated to storing **confidential data** (e.g. credentials).
 - That data isn't stored in plain text → **stored in a Base64 encoded format**.
4. **Base64 encoding** format is **not enough**:
 - Because **Kubernetes isn't encrypted by default**.
 - Consequence: Have to **use another third-party tool** to encrypted **secrets**.
5. In a nutshell: **Secrets** store **sensitive data** (credentials, tokens) in Base64-encoded format.
 - Usually **encrypted by third-party tools** since **Kubernetes doesn't encrypt them by default**.

9. Volumes and Persistence

1. Database storage in Kubernetes: we use **Volumes** component. Why ?

→ Since Kubernetes doesn't manage **persistent** storage.

→ Because storing data into database service → if pod (or database container inside) res
⇒ all of data inside this pod **will be lost**.

2. How does Volume component work ?

→ Attaches **physical storage** like a hard drive to **your pod**.

→ This physical storage could be **mounted locally** or **from a remote server**.

→ "**Local storage**" means "*in the same node/server of the Kubernetes cluster*" (the one where the DB service is located).

→ "**From a remote server**" means "*outside of the Kubernetes cluster*".

→ **Remote storage** = **Cloud storage** or even **On-premise storage on another machine**.

3. **Volume** component = **not a part of the Kubernetes cluster** → just have a reference on it.

4. Now: even if database component is restarted (the container or the pod)

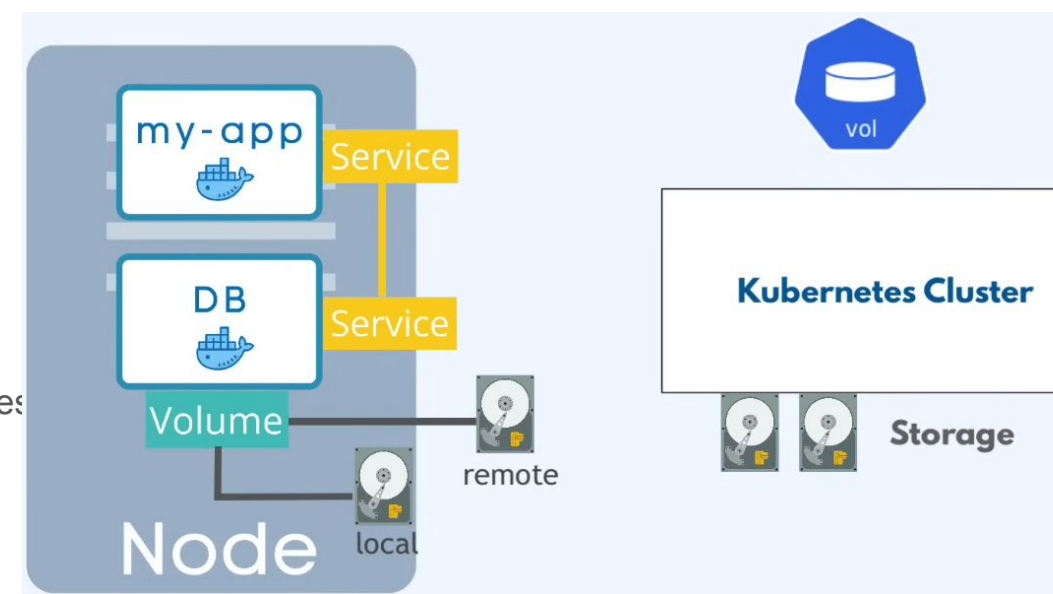
→ All of the data attached with that component **will be persisted**.

5. In a nutshell: K8s doesn't manage **any data storage persistence**.

→ Volumes are used to attach **local** or **external storage** (cloud or on-premise remote server).

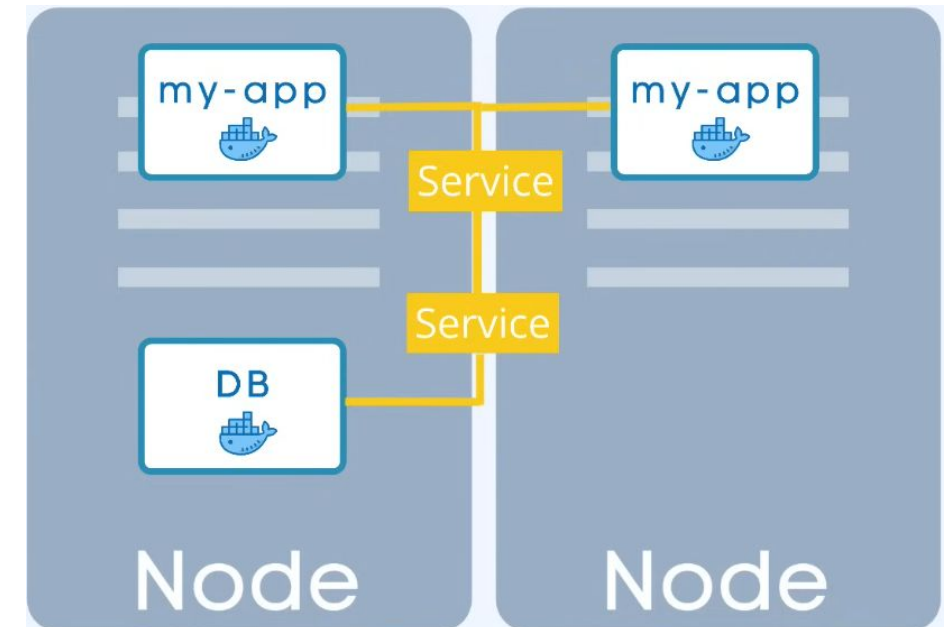
→ This ensures data persists even if a pod restarts.

→ Developers or admins are responsible for managing and configuring these backend / replication storages.



10. Replication

1. When a pod (of your app container) crashes down→downtime→a very bad thing in production.
 - To prevent downtimes→in K8s we benefit from a **distributed system**→**pods** are **replicated across nodes**.
 - Replicating everything on multiple servers: we **replicate each pod** from each service.
 - e.g.: the app pod & the database pod are replicated on **different nodes** of the **distributed system**.
2. **Replica** is **connected** to the **same Service**.
 - Consequence: having a replica / clone of our app which is **running with the same service**.
 - Reminder: Service = permanent / persistent IP address, with a **DNS name**:
 - Don't have to constantly adjust the endpoint when the pod dies.



10'. Deployments

1. **Service** = also a **Load Balancer**:

- Catch & forward requests to whichever part is least busy in the Cluster.
- *i.e* to the pod which is available.

2. To create the **replica** of a pod:

- Don't have to create that replica.
- Just create the **blueprint** of the pod (**number of replicas** for that pod you want to run).
- This component, this blueprint is called **Deployment**.

3. In practice:

- Never create pods / work with pods.
- **Only work with Deployments** (specifying inside the number of replicas you want for a pod).
- Consequence: Can **scale up** or **scale down** as you wish.

4. As Pods are a layer of abstraction on top of Containers → **Deployments** = another **level of abstraction** on **top of the Pods**.

```
io.k8s.api.apps.v1.Deployment (v1@deployment.json)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp-deployment
  labels:
    app: webapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
```

Deployment part

Pod part

10'. Deployments

5. As a consequence:

- If an instance of your pod dies ⇒ **Service** will instantly **forward** the user request to **another replica pod**.
- So the Service = **still be accessible** / available for the user.

6. Similarly we need **replica of our database pod**.

- A little bit different because we **can't have a strict replica of our database**.
- With data clones / replicas of our DB: we always need to **have access to the same database data storage** (consistency).

7. In a nutshell:

- To prevent downtime, **pods** are **replicated across nodes**.
- Services distribute traffic via **load balancing** to **available** replicas.
- Replication is managed by a **Deployment**, which defines **how many replicas of a pod** to run.
- Deployments are **abstractions on top of pods**, enabling easy scaling and **automatic recovery** if a pod fails.

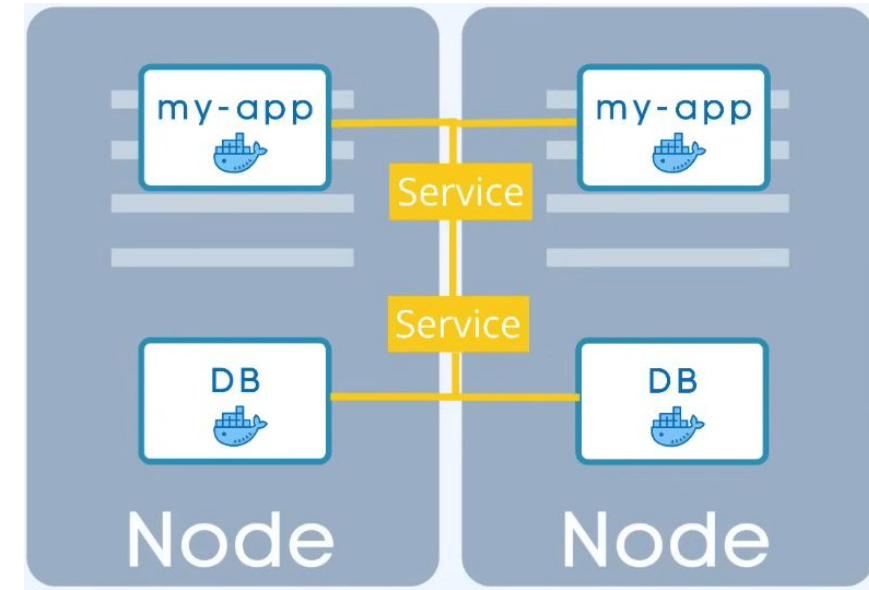
11. StatefulSets

1. Need additional mechanism which decides which pod is currently writing / vs reading on the database:

- To ensure **consistency** of the database.
- This mechanism is offered by another component called **StatefulSet**.
- This component = meant specifically for applications like databases.

2. **Stateful** apps / **databases** such MongoDB, MySQL, Elasticsearch, etc:

- StatefulSet = for stateFUL apps or databases
- **Should be created** with **StatefulSets**, not with deployment component,
- which manage replicas that must **share the same persistent storage**,
⇒ *i.e.* **ensure** read/write **consistency between replicas**.



3. Working with stateful components / DB is more complex than working with Deployments.

- So, this is common to **host databases outside** the Kubernetes cluster.

4. Pretty common to **just** have **Deployment** or **Stateless** apps **inside** the K8s cluster.

- Deployment = for stateLESS apps
- As you can play with different replicas with no major issue,
- which **replicates & scales** correctly inside the Kubernetes cluster.
- & communicate with external databases.

12. Master Node and API Server

1. All configuration in Kubernetes cluster: goes through a **Master Node**, with a process called **API Server**.

→API Server = the **only entry point** (of the Control Plane) in the Kubernetes cluster.

2. Kubernetes **Client**: UI (e.g. dashboard), CLI, or an API (with a script / curl command calling endpoint).

3. All of these client components:

→**Talk to the API server** (using declarative YAML or JSON files).

→Send their **Configuration Request** (YAML/JSON format) **to the API Server**.

→This request **corresponds** to a **configuration file**.

→e.g. telling K8s to create **two replicas pods** which **labels:app: myapp**,
with inside each pod a **containers:-name: myapp**, based on an **image:myimage**.

4. Configuration requests in K8s are **declarative** = declare what you want to see as an outcome.

→Kubernetes tries to **meet these requirements**.

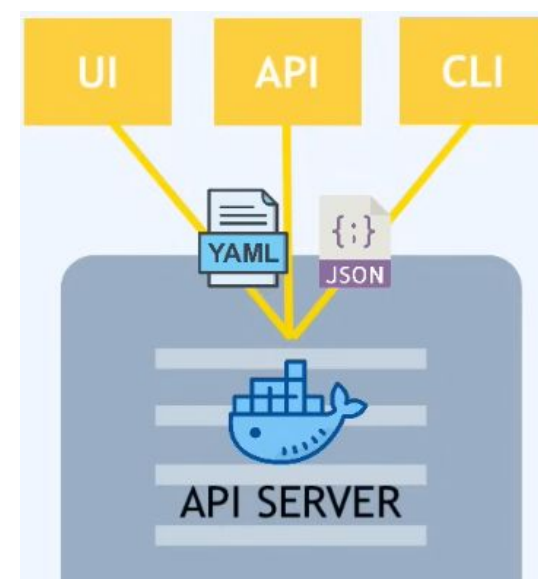
→ Continuously reconciles the actual state with the desired one (**self-healing**).

→e.g. if we configure two replicas of a single pod and one of the pods dies:

→Kubernetes will see that now we only have one pod;

→which it's *not the desired configuration*.

→So it will work on **recovering** the **desired state** of two pods.



```
io.k8s.api.apps.v1.Deployment (v1@deployment.json)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
    app: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          # image: myrepo/myimage:tag
          ports:
            - containerPort: 3000
```

Pod part

13. Configuration files

1. Each configuration file in Kubernetes has three parts.

- First part: the **Metadata** part (of the component you create).
- Second part: the **Specification** part.
 - Corresponds to every configuration / **specification** you **want** to apply to that component.
 - Attributes of the **spec:** section are **specific** to the **kind** of component you create.
 - > e.g. **spec:replicas:**, **spec:selector:matchLabels:**, **spec:template:**, etc.
- Third part: the **Status** part.
 - Status: which is **automatically generated** & added by **Kubernetes**.
- (Fourth part): **kind:** e.g. Deployment (at the top file).

2. What is the status part (as shown before) ?

- Kubernetes will always **compare** the current / **actual status** vs **desired status**.
- e.g. having 3 pod's replicas on your configuration & currently having just 2 running pods
- This is the basis of the **self-healing** feature Kubernetes has.
- Will **update** the status **continuously** to **match** the **desired** configuration.

```
io.k8s.api.apps.v1.Deployment (v1@deployment.json)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels: ...
spec:
  replicas: 2
  selector: ...
  template: ...

io.k8s.api.apps.v1.Deployment (v1@deployment.json)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
    app: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers: ...

status:
  availableReplicas: 1
  conditions:
  - lastTransitionTime: "2020-01-24T10:54:59Z"
    lastUpdateTime: "2020-01-24T10:54:59Z"
    message: Deployment has minimum availability.
    reason: MinimumReplicasAvailable
    status: "True"
    type: Available
  - lastTransitionTime: "2020-01-24T10:54:56Z"
    lastUpdateTime: "2020-01-24T10:54:59Z"
    message: ReplicaSet "nginx-deployment-7d64f4b"
    reason: NewReplicaSetAvailable
    status: "True"
    type: Progressing
  observedGeneration: 1
  readyReplicas: 1
  replicas: 1
  updatedReplicas: 1
```

14. etcd

Where does Kubernetes keep that **status data** to **continuously update** it ?

- Possible with "**etcd**" component.
- etcd component keeps a track of any other Kubernetes component.
- etcd component stores all cluster state and configuration data.
 - Enabling Kubernetes to track and update the system's current status.
 - To match the desired configuration.

15. Infrastructure as Code

1. Format of the **configuration files** in Kubernetes is **YAML**.
2. YAML configuration files are usually **stored** in the project's root or in a dedicated Git repository.
 - Since your Deployment, Service or Ingress configuration will be applied to your application,
 - it's a good practice to store this configuration files in your application folder.
3. This ensures that Deployments, Services, and Ingress definitions are versioned alongside the application – aligning with the **Infrastructure as Code** approach.

16. Minikube and kubectl

1. **Minikube** is a lightweight single-node Kubernetes cluster for local testing, containing both master and worker components, with a Docker runtime preinstalled.

2. **kubectl** is the main command-line tool to interact with any Kubernetes cluster (not just Minikube).

It communicates with the API server to create, update, or delete resources like pods, services, and deployments.

17. Labels and Deployments

1. In Kubernetes, all components can be assigned **key-value labels**, which act as additional identifiers beyond the component **name**.
2. Pods always have **unique names** (auto-generated by K8s), but **replicas** of the same pod share **the same label**.
⇒ Can identify **all replicas of a pod** using this **unique & common label** they share (e.g. app: webapp).
3. **Labels** are **mandatory for pods** and are used to **identify**, group, and manage **replicas**, while for **other components** (Deployment, ConfigMap, etc.) they are **optional** but strongly recommended as a best practice.
4. To **deploy all the replicas of the unique pod** → give it to Kubernetes the **label name** (**pods** → **template:metadata:labels:**) / not the name of each pod.
 - Because uniqueness contained into the label, not into the name of each replica.
 - For Pods, **template:metadata:labels:** section of the pods = required, not optional.
 - For Deployment, **selector:matchLabels:** = also required.
 - **template:metadata:labels:** in **Pods** ⇔ **selector:matchLabels:** in **Deployment**

```
io.k8s.api.apps.v1.Deployment (v1@deployment.json)
  apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: webapp-deployment
    labels:
      app: webapp
  spec:
    replicas: 1
    selector:
      matchLabels:
        app: webapp
    template:
      metadata:
        labels:
          app: webapp
```

Deployment part

Pod part

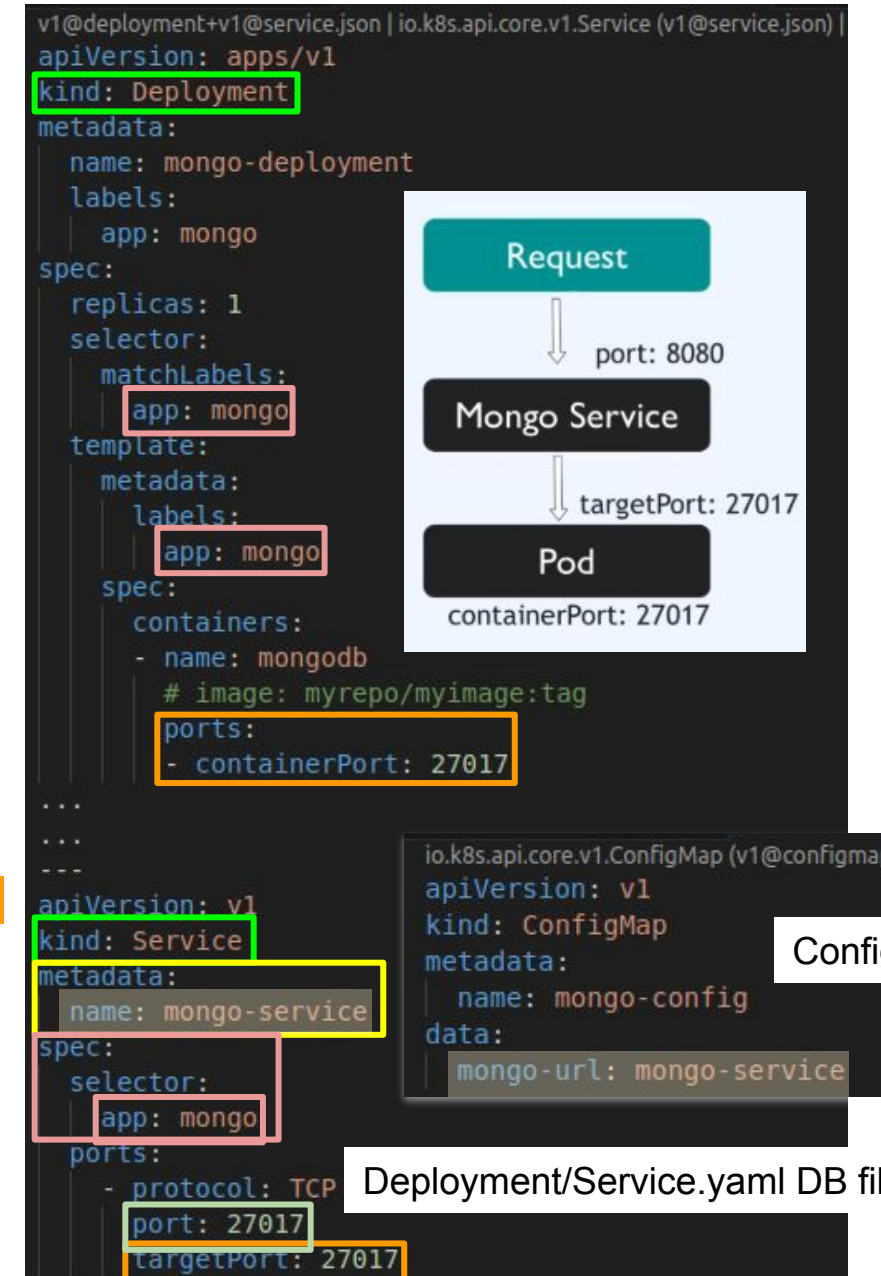
18. Deployment Selectors and Pod Matching

1. Kubernetes determines **which pods belong to a Deployment** using **selector:matchLabels:**, defined in the Deployment **spec:**.
 - All pods created by a Deployment must share same **label** defined in the Deployment **selector:matchLabels:** section.
e.g. `app: webapp`
 - Which Pods belongs to Deployment ?
 - **Replicas** = how many Pods you want to create
2. A common convention is to use **app: <application-name>** as the label key-value pair.
3. Databases should not be scaled using Deployments but with **StatefulSets**.

```
io.k8s.api.apps.v1.Deployment (v1@deployment.json)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp-deployment
  labels:
    app: webapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
```

19. Services, Ports, and Target Ports

1. → "---" = You can have multiple YAML configurations within 1 file.
2. A Service defines a stable endpoint (defined into the **metadata:** section, e.g. **mongo-service**) used by other components, such as ConfigMaps.
 - **name: <arbitrary_name>**
3. The Service **spec:selector:** section determines **which pods it targets**, while **spec:ports:targetPort:** corresponds to the **container port inside the pod**.
 - **spec:selector:** selects **pods to forward** the requests to.
 - It defines **which Pods belong** to that **Service**.
 - **targetPort:** = where the application in the pod is accessible/lives.
4. **Requests reach Service** through **port:** & are forwarded to pods through **targetPort:**.
 - **port:** = **Service port** (for the **request / external user**)
(to forward the request to the Service).
 - **targetPort:** = **containerPort of Deployment**
(internal) = used by Service to forward to pods where the app lives.
 - Often both values are set identically.



20. Environment Variables and Secrets (example of a Database conf)

1. Environment variables are configured using the **env** field in the container specification.
2. Instead of hardcoding values, Kubernetes allows referencing **Secrets** using **valueFrom:**, pulling sensitive values (e.g. **username**, **password**) from a **secrets.yaml** file at runtime.
 - Reference Secret Data here instead of hardcode sensitive data.
 - secrets.yaml config file shoudn't be checked into Git repo.

```
containers:
- name: mongodb
  image: mongo:5.0
  ports:
  - containerPort: 27017
  env:
  - name: MONGO_INITDB_ROOT_USERNAME
    valueFrom:
      secretKeyRef:
        name: mongo-secret
        key: mongo-user
  - name: MONGO_INITDB_ROOT_PASSWORD
    valueFrom:
      secretKeyRef:
        name: mongo-secret
        key: mongo-password
---
apiVersion: v1
kind: Service
metadata:
  name: mongo-service
```

Secrets.yaml file

Deployment/Service.yaml DB file

21. Application-to-Database Configuration

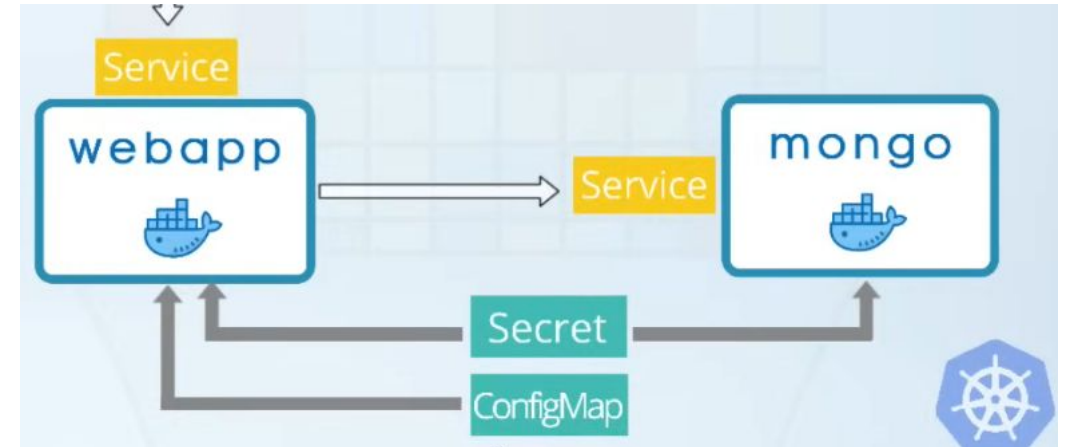
1. Applications need database connection details such as **endpoint**, **username**, **password**.

→ Webapp needs information about:

- DB endpoint (ref from ConfigMaps)
- DB username (ref from Secrets)
- DB password (ref from Secrets)

2. These values are injected as environment variables from **ConfigMaps** (for **endpoints**) and **Secrets** (for credentials), exactly as done for the database Deployment itself.

→ **configMapKeyRef** instead of **secretKeyRef**.



```
spec:
  containers:
  - name: webapp
    # image: myrepo/myimage:tag
    ports:
    - containerPort: 3000
    env:
    - name: USER_NAME
      valueFrom:
        secretKeyRef:
          name: mongo-secret
          key: mongo-user
    - name: USER_PWD
      valueFrom:
        secretKeyRef:
          name: mongo-secret
          key: mongo-password
    - name: DB_URL
      valueFrom:
        configMapKeyRef:
          name: mongo-config
          key: mongo-url
```

```
io.k8s.api.core.v1.ConfigMap (v1@configmap.json)
apiVersion: v1
kind: ConfigMap
metadata:
  name: mongo-config
data:
  mongo-url: mongo-service
```

ConfigMaps.yaml

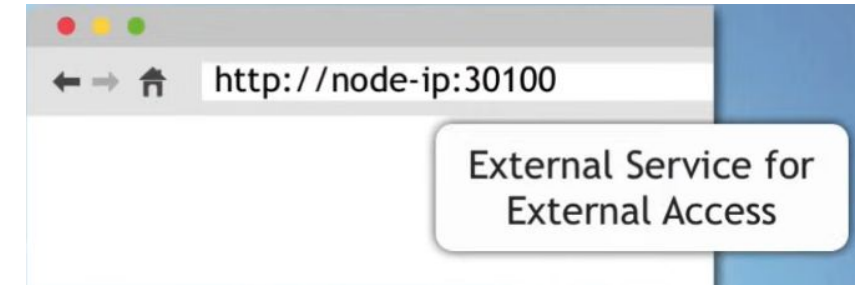
Deployment/Service.yaml Webapp file

22. Clean and Secure Configuration

1. No environment variables are hardcoded in Deployment YAML files; **all values come from ConfigMaps or Secrets**.
2. This makes configuration way **cleaner**, **safer**, and **easier** to maintain, since **changing a value only requires updating the ConfigMaps or Secrets** - without modifying multiple Deployment files.

23. External Access with NodePort

1. Final goal: making this application accessible from the browser (from the outside).
 - (i.e. being able to type in some URL and access our web application).
 - For doing that, we have **external services**.
2. By **default, Services are internal (ClusterIP)**.
 - To **expose an application externally**, the **Service type** can be set to **NodePort**.
3. In order to define it as an external service we just have to define a new **type:** with a value.
 - e.g. we can choose **NodePort** which is an **external service type**.
 - The NodePort type is a port on **which the application will be accessible**.
 - We also have to define a new port into the **spec:ports:** section called **nodePort:**.
4. A NodePort type opens a port on **each cluster node**, making the application accessible via **<any-node-ip>:<node-port>** from outside the cluster.
 - nodePort** exposes the Service on **each node's IP** at a static port.
 - **nodePort** value must be between 30000 and 32767.
5. **type: NodePort for development only.**
 - Better to use **LoadBalancer + Ingress + ClusterIP** (for all services, all internal).



```
apiVersion: v1
kind: Service
metadata:
  name: webapp-service
spec:
  type: NodePort
  selector:
    app: webapp
  ports:
    - protocol: TCP
      port: 3000
      targetPort: 3000
      nodePort: 30100
```

Service.yaml Webapp file

24. Deployment Order and Dependencies

1. **ConfigMaps** and **Secrets** must be created before Deployments, as Deployments reference them during creation.
2. A logical order is required: first ConfigMaps and Secrets, then the database Deployment, and finally the application Deployment that depends on this database.
 - DB Service `metadata:name: mongo-service`
 - ConfigMaps `data:mongo-url: mongo-service`
 - Webapp Deployment Pods `spec:template:spec:containers:env:-name:valueFrom:configMapKeyRef:key: mongo-url`

:

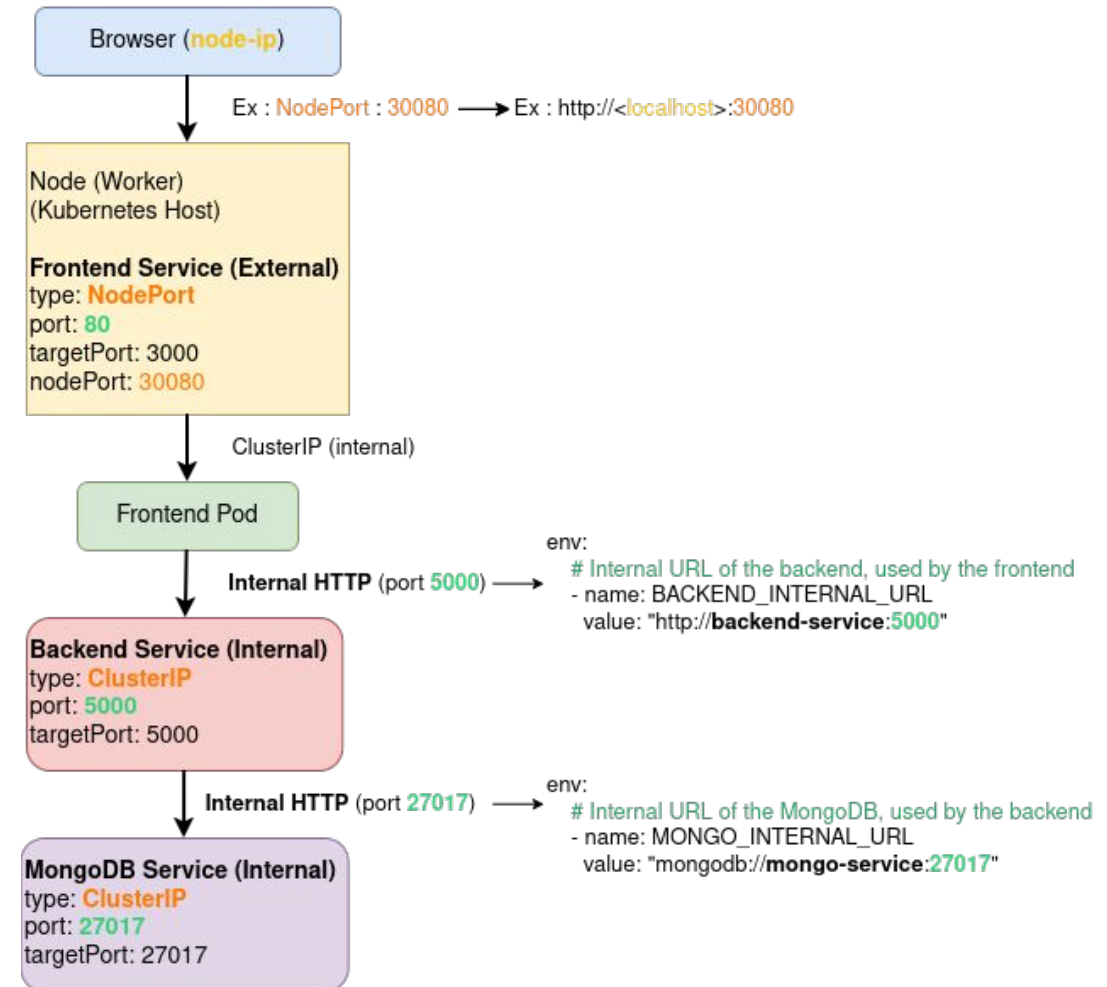
25. Focus on a typical Kubernetes architecture

Dev version (simple, fast, no Ingress)

Context

- No Ingress
- No Load Balancer
- Just a NodePort to expose the **External Frontend Service** (for a quick test)
- Backend + Metrics = **Internal**
- Mongo = **Internal**
- K8s resolves “**mongo-service**” → **internal DNS / “IP address”** of the MongoDB service.
- The backend automatically contacts MongoDB with:
mongodb://<service-name>:<port>
 - **mongodb://** → MongoDB client protocol.
 - **mongo-service** → **DNS name** of the cluster's internal MongoDB service.
 - **:27017** → port exposed by the MongoDB service.

DEV - With ports diagram



25. Focus on a typical Kubernetes architecture

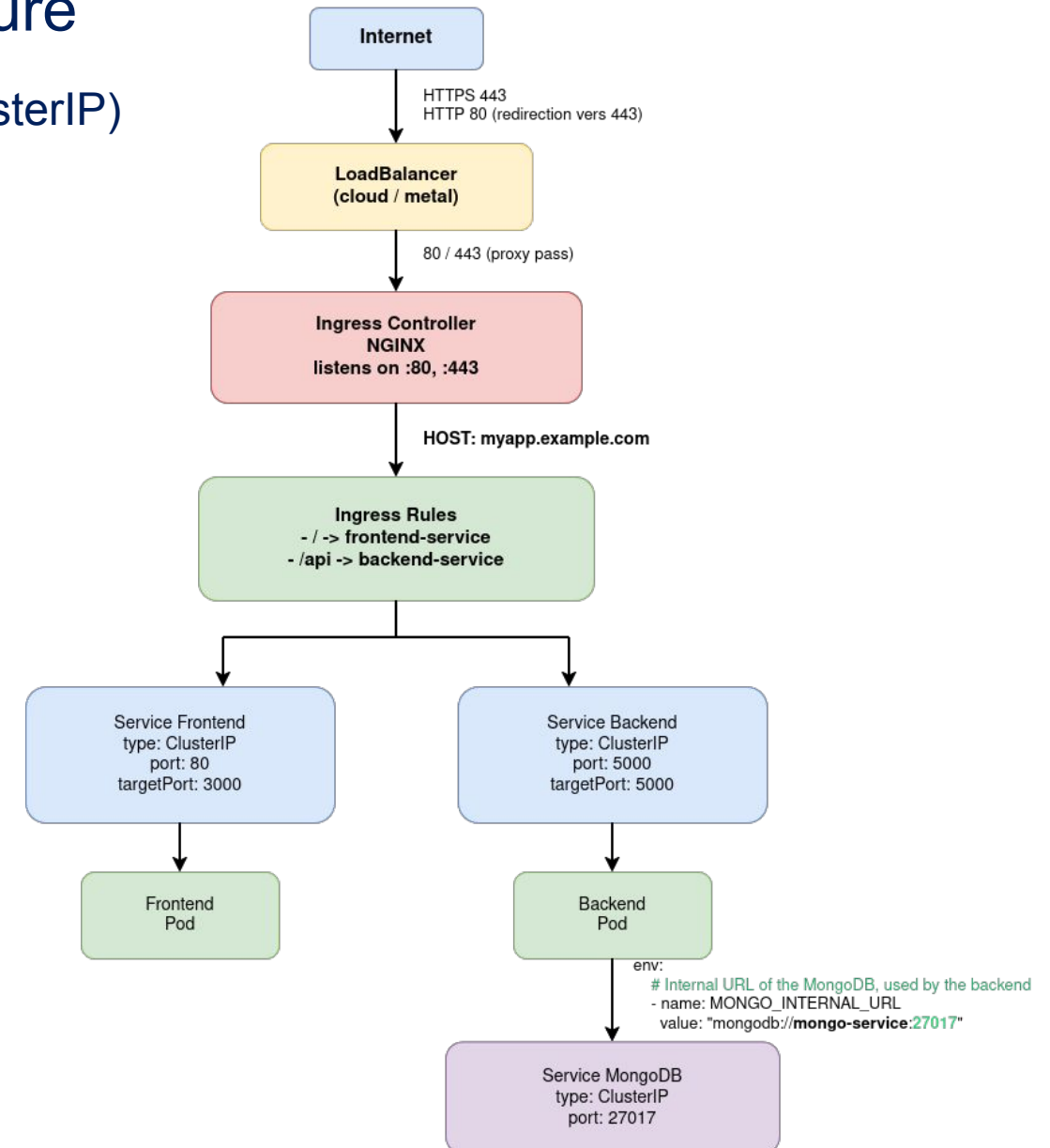
Prod version (Ingress, no external services, only internal ClusterIP)

Nginx = Ingress Controller = **reverse proxy** in the cluster
(Nginx / Traefik / HAProxy).

Context

- When accessing the frontend from outside the cluster:
 - **Just call the LoadBalancer on ports 80/443.**
 - **443 (HTTPS) or 80 (HTTP).**
- Then:
 - 443/80 (client browser)
 - → Load Balancer 443/80
 - → Ingress Controller 443/80
 - → Frontend Service (**internal** port 80)
 - → Frontend Pod (port 3000)
- For internal Kubernetes *communication* (between components):
 - Frontend Service: **80**→**3000**
 - Backend Service: e.g. **5000**→**5000**
 - Mongo Service: **27017**→**27017**

PROD - With LoadBalancer & Ingress diagram



25. Focus on a typical Kubernetes architecture

Prod version (Ingress, no external services, only internal ClusterIP)

Ingress Rules
- / -> frontend-service
- /api -> backend-service

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
  namespace: my-namespace
spec:
  ingressClassName: nginx
  rules:
    # Frontend & Backend API (public access)
    - host: myapp.example.com
      http:
        paths:
          # Frontend
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80
          # Backend API (public access)
          # This is NOT the backend-service which has its own public DNS
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: backend-service
                port:
                  number: 5000
```